Ensemble Learning

Wenbin Wang

Advisor: Prof. Jinhua Zhao & Dr. Xinyu Chen

School of Information Science and Technology
ShanghaiTech University, Shanghai, China

MIT-UF-NEU 2025 Summer Research Camp

July 22th, 2025

Outline

- Introduction
- Types & Popular Algorithms
- Challenges & Current Research Progress
- Future Directions
- Summary

Outline

- Introduction
- Types & Popular Algorithms
- Challenges & Current Research Progress
- Future Directions
- Summary

Reference books and materials

- `http://www.scholarpedia.org/article/Ensemble_learning`
- `http://en.wikipedia.org/wiki/Ensemble_learning`
- `http://en.wikipedia.org/wiki/Adaboost`
- Rudin video: `http://videolectures.net/mlss05us_rudin_da/`

What is Ensemble Learning?

Ensemble learning is a machine learning technique that combines multiple models to create a more accurate and robust model than any individual model could achieve on its own. It leverages multiple machine learning algorithms collectively to address classification or regression problems. This is done by training several models on the same/independent datasets and then aggregating their predictions to make a final decision.

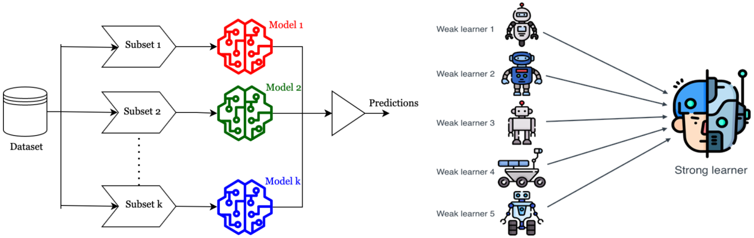


Figure 1: Ensemble Learning[1]

---

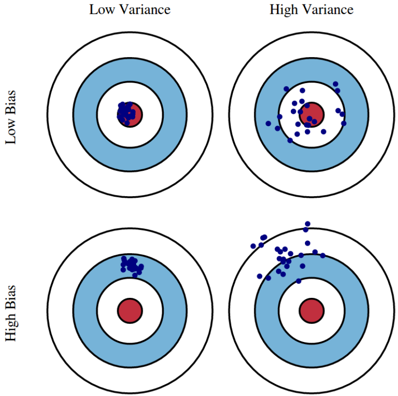
[1] https://images.app.goo.gl/qKHd7LhBxrsUGvhG9 & https://images.app.goo.gl/sUYxZxKfiC7zzGVE7

Why Do We Need Ensemble Learning?

Rationale:

- **No Free Lunch Theorem: There is no algorithm that is always the most accurate** `http://en.wikipedia.org/wiki/No_free_lunch_in_search_and_optimization`
- **Generate a group of base-learners which when combined has higher accuracy**
- **Each algorithm makes assumptions which might be or not be valid for the problem at hand or not.**
- **Different learners use different**
  - **Algorithms**
  - **Hyperparameters**
  - **Representations (Modalities)**
  - **Training sets**
  - **Subproblems**

No Free Lunch Theorem

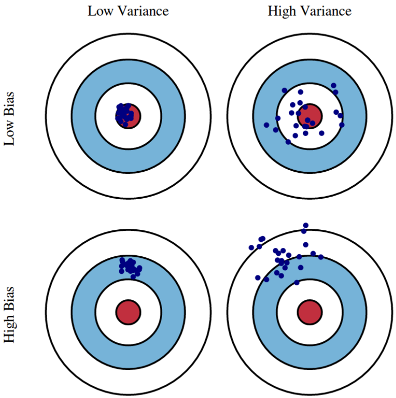


Bias: Quantify how much on an average are the predicted value different from the actual value[2].

---

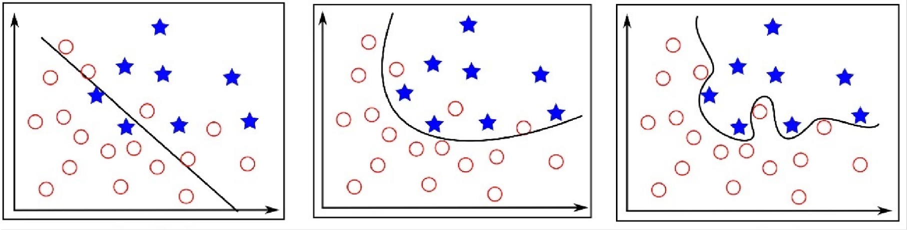
[2] Image source: G. M. Tina, C. Ventura, S. Ferlito, and S. De Vito, A State-of-Art-Review on Machine-Learning Based Methods for PV, Applied Sciences, vol. 11, no. 16, p. 7550, Aug. 2021, doi: 10.3390/app11167550.

No Free Lunch Theorem



Variance: Quantifies how much of the predictions made on the same observation different from each other. High variance can cause an algorithm to model the random noise in the training data, rather than the intended outputs.

No Free Lunch Theorem



High Bias (Under-fitting)

Low Bias, Low Variance

High Variance (Over-fitting)

Image source: `https://vitalflux.com/bagging-classifier-python-code-example/`

General Idea

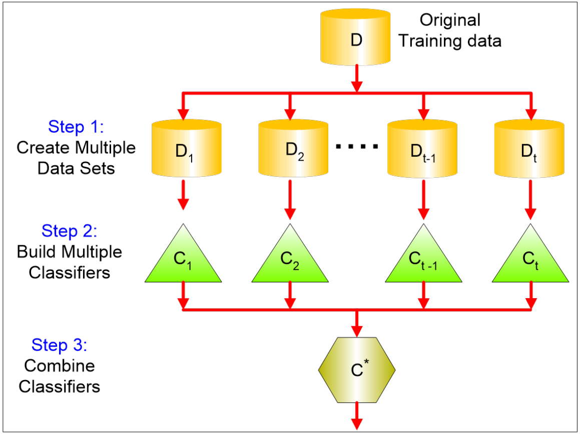


Figure 2: Ensemble Learnig

Why does it work?

- Suppose there are 25 base classifiers
  - Each classifier has error rate, $\epsilon = 0.35$
  - Assume classifiers are independent
  - Probability that the ensemble classifier makes a wrong prediction:

$$\sum_{i=13}^{25} \left(\begin{array}{c} 25 \\ i \end{array} \right) \epsilon^{i} \left(1 - \epsilon\right)^{25-i} = 0.06$$

Why are Ensemble Successful?

- **Bayesian perspective:**

$$P\left(C_i \mid x\right)=\sum_{\text{all models } M_j} P\left(C_i \mid x, M_j\right) P\left(M_j\right)$$

  - If d_j are independent

$$\operatorname{Var}(y)=\operatorname{Var}\left(\sum_j \frac{1}{L} d_j\right)=\frac{1}{L^2} \operatorname{Var}\left(\sum_j d_j\right)=\frac{1}{L} \operatorname{Var}\left(d_j\right)$$

    Bias does not change, variance decreases by L
  - If dependent, error increases with positive correlation

$$\operatorname{Var}(y)=\frac{1}{L^2} \operatorname{Var}\left(\sum_j d_j\right)=\frac{1}{L^2}\left[\sum_j \operatorname{Var}\left(d_j\right)+2 \sum_j \sum_{i<j} \operatorname{Cov}\left(d_i, d_j\right)\right]$$

The Advantages of Ensemble Learning

Ensemble learning offers numerous advantages. Here, we will examine its most compelling benefits and explain why its such a powerful tool.

- **Reducing Overfitting**
- **Improving Accuracy**
- **Handling Noisy Data**
- **Handling Imbalanced Data**
- **Handling Large Datasets**
- **Versatility**

Outline

- Introduction
- Types & Popular Algorithms
- Challenges & Current Research Progress
- Future Directions
- Summary

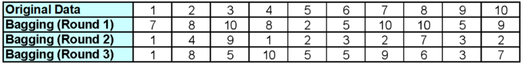
Types & Popular Algorithms

There are different types of Ensemble Learning techniques which differ mainly by the type of models used (homogeneous or heterogeneous models), the data sampling (with or without replacement, k-fold, etc.), and the decision function (voting, average, meta model, etc). Therefore, Ensemble Learning techniques can be classified as:

- **Bagging(Bootstrap Aggregation)**
- **Boosting**
  - **AdaBoost**
  - **Gradient Boosting**
  - **XGBoost**
- **Stacking**
- **Voting**
- **Other ensemble learning approaches**
  - **Bucket of Models**
  - **Bayesian Model Averaging**

Bagging

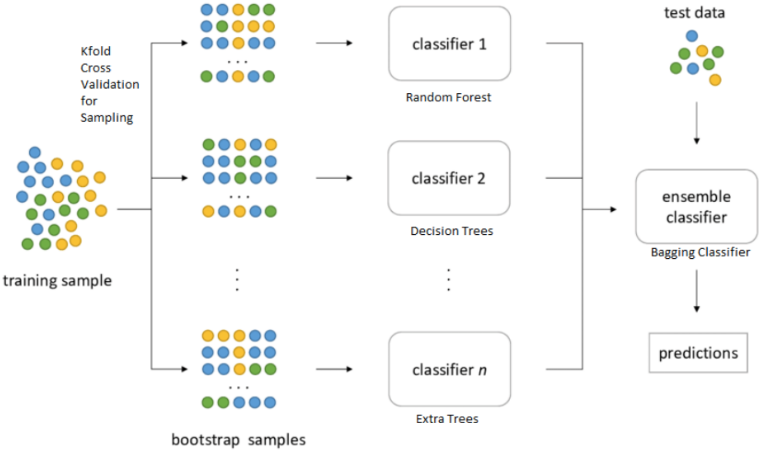
- Use bootstrapping to generate L training sets and train one base-learner with each [1]
- Use voting (Average or median with regression)
- Unstable algorithms profit from bagging
- Example:
    - Sampling with replacement



Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

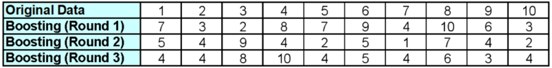
-
    - Build a classifier on each bootstrap sample
    - Each sample has probability $\left(1-\frac{1}{n}\right)\cdot n$ of being selected

Bagging


Kfold Cross Validation for Sampling
training sample
bootstrap samples
classifier 1
Random Forest
classifier 2
Decision Trees
classifier n
Extra Trees
test data
ensemble classifier
Bagging Classifier
predictions

Boosting

- An iterative procedure to adaptively change the distribution of training data by focusing more on previously misclassified records
  - Initially, all N records are assigned equal weights
  - Unlike bagging, weights may change at the end of the boosting round
- Records that are wrongly classified will have their weights increased
- Records that are classified correctly will have their weights decreased
- Example:



Original Data	1	2	3	4	5	6	7	8	9	10
Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

  - Example 4 is hard to classify
  - Its weight is increased, therefore it is more likely to be chosen again in subsequent rounds

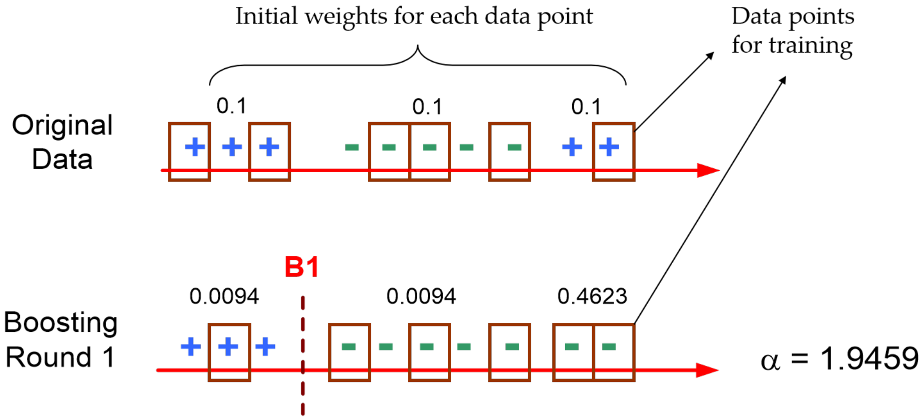
AdaBoost

Algorithm 1: AdaBoost Ensemble Learning

```
Input  : Initial dataset $D_1$ with $N$ examples, number of iterations $k$
Output: Ensemble classifier $H$

Initialize equal weights: $w_j^{(1)} \leftarrow \frac{1}{N}$ for all $j = 1$ to $N$
for $i \leftarrow 1$ to $k$ do
    Train classifier $C_i$ on $D_i$ using current weights $\mathbf{w}^{(i)}$
    Compute classifier weight: $\alpha_i \leftarrow \frac{1}{2} \ln \left( \frac{1-\epsilon_i}{\epsilon_i} \right)$    // $\epsilon_i$ is the weighted error of $C_i$
    Update weights for each example $j$:
        $w_j^{(i+1)} \leftarrow w_j^{(i)} \cdot \exp\left(-\alpha_i \cdot y_j \cdot C_i(\mathbf{x}_j)\right)$
    Normalize weights: $\mathbf{w}^{(i+1)} \leftarrow \mathbf{w}^{(i+1)} / \sum_j w_j^{(i+1)}$
    Create $D_{i+1}$ by weighted sampling from $D_i$ with weights $\mathbf{w}^{(i+1)}$
end
Construct ensemble classifier: $H(\mathbf{x}) = \left( \sum_{i=1}^{k} \alpha_i C_i(\mathbf{x}) \right)$
```

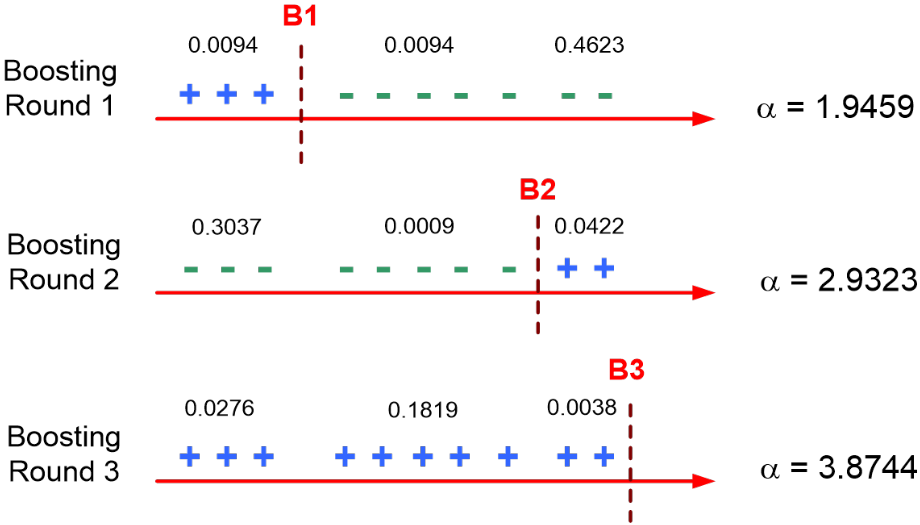
Illustrating AdaBoost



Video Containing Introduction to AdaBoost:

`http://videolectures.net/mlss05us_rudin_da/`

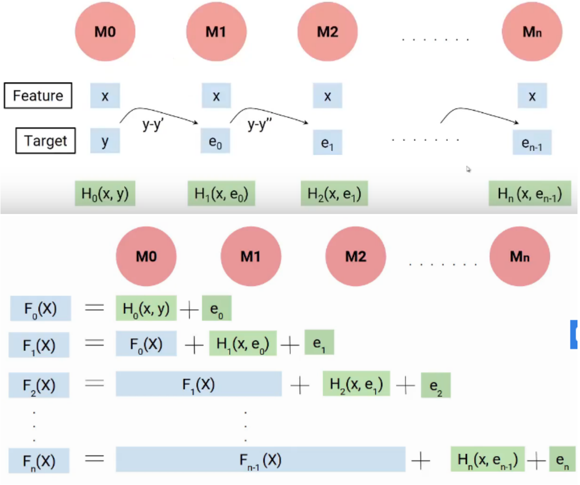
Illustrating AdaBoost



Gradient Boosting

- Often used with decision trees and regression
- Common winner in task competitions
- Train first model F_1 with the basic training set
- Train next model h creating updated ensemble model $F_{m+1} = F_m + h$
- But, train h using the residual/error (the difference between the target and the current output of F_m)
    - For h, change training instance (x, y) to $(x, y - F_m(x))$
    - Each new model learns to output and cancel the remaining error from the previous model, leaving less error with each model
    - Learning focuses on instances where the latest F_m has higher error
- Also learns each models weighting coefficient γ with gradient descent to minimize chosen loss function (SSE common)
- Once trained, F_m no longer changes, and we keep adding new hs until remaining error is almost gone or test error begins to increase

Gradient Boosting



Gradient Boosting

$$F_{n+1}(X) = F_n(X) + \gamma_n H(x, e_n)$$

$$\begin{aligned}
F_0(X) &= \gamma_0 H_0(x, y) + e_0 \\
F_1(X) &= F_0(X) + \gamma_1 H_1(x, e_0) + e_1 \\
F_2(X) &= F_1(X) + \gamma_2 H_2(x, e_1) + e_2 \\
&\vdots \\
F_n(X) &= F_{n-1}(X) + \gamma_n H_n(x, e_{n-1}) + e_n
\end{aligned}$$

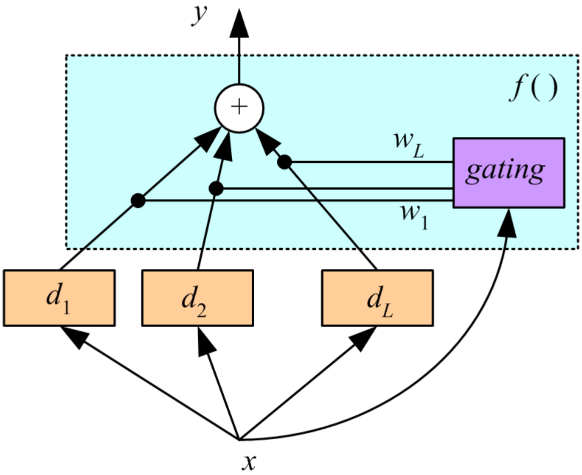
- How to combine? Each models weighting/voting coefficient γ_i is learned with gradient descent to minimize loss as models are created, then frozen

An Example of Gradient Boosting

- Fit a shallow regression tree T_1 to the data
    - the first model is $M_1 = T_1$
    - The shortcomings of the model are given by the negative gradients.
- Fit a tree T_2 to the negative gradients
    - The second model is: $M_2 = M_1 + \eta \cdot \gamma_2 \cdot T_2$
    - η is a learning rate to encourage more models
    - γ_i is optimized, then frozen, so that M_i best fits the data
- Continue adding models (trees) until stopping criteria met
- The final model is $M_{\text{final}} = M_{\text{final}-1} + \eta \cdot \gamma_{\text{final}} \cdot M_{\text{final}}$

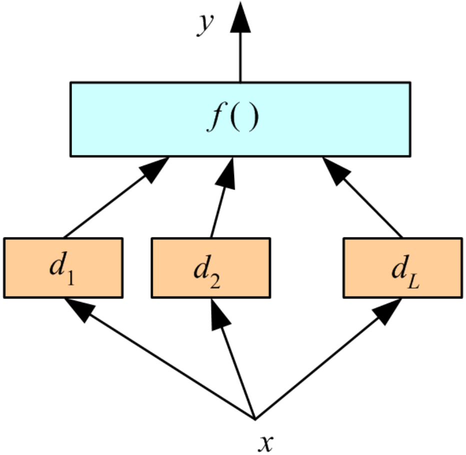
Voting

Voting where weights are input-dependent (gating) [2]



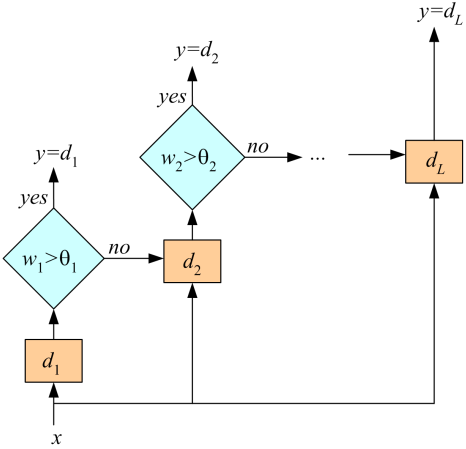
Stacking

Combiner $f()$ is another learner [3]



Cascading

Use d_j only if the preceding ones are not confident
Cascade learners in order of complexity



Outline

- Introduction
- Types & Popular Algorithms
- Challenges & Current Research Progress
- Future Directions
- Summary

What is the Main Challenge for Developing Ensemble Models?

- The main challenge is not to obtain highly accurate base models, but rather to obtain base models that make different kinds of errors.
  - For example, if ensembles are used for classification, high accuracies can be accomplished if different base models misclassify different training examples, even if the base classifier accuracy is low. Independence between two base classifiers can be assessed in this case by measuring the degree of overlap in training examples they misclassify ($|A \cap B| \,/\, |A \cup B|$)—more overlap means less independence between two models.
- Interpretability: Harder to interpret than single models.
- Model Selection: Choosing base learners and ensemble strategy.
- Computational Cost: Ensembles require more resources.

Outline

- Introduction
- Types & Popular Algorithms
- Challenges & Current Research Progress
- **Future Directions**
- Summary

Frame Title

Outline

- Introduction
- Types & Popular Algorithms
- Challenges & Current Research Progress
- Future Directions
- Summary

Summary

- Ensemble approaches use multiple models in their decision making. They frequently accomplish high accuracies, are less likely to over-fit and exhibit a low variance. They have been successfully used in the Netflix contest and for other tasks. However, some research suggest that they are sensitive to noise (`http://www.phillong.info/publications/LS10_potential.pdf`).
- The key of designing ensembles is diversity and not necessarily high accuracy of the base classifiers: Members of the ensemble should vary in the examples they misclassify. Therefore, most ensemble approaches, such as AdaBoost, seek to promote diversity among the models they combine.
- The trained ensemble represents a single hypothesis. This hypothesis, however, is not necessarily contained within the hypothesis space of the models from which it is built. Thus, ensembles can be shown to have more flexibility in the functions they can represent. Example: `http://www.scholarpedia.org/article/Ensemble_learning`
- Current research on ensembles centers on: more complex ways to combine models, understanding the convergence behavior of ensemble learning algorithms, parameter learning, understanding over-fitting in ensemble learning, characterization of ensemble models, sensitivity to noise.

Reference

- L. Breiman, "Bagging predictors," *Machine learning*, vol. 24, no. 2, pp. 123–140, 1996.
- R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural computation*, vol. 3, no. 1, pp. 79–87, 1991.
- D. H. Wolpert, "Stacked generalization," *Neural networks*, vol. 5, no. 2, pp. 241–259, 1992.